高级程序设计实验报告

实验七

题目:	实验六	指导教师:	冯国栋
姓名:	林楗棋	学号:	24363050
专业班号:	202513912	学院:	智能工程学院

目录

1 实验目的与设计思路 **2**

1.1 实验目的 2

1.2 设计思路 2

2 核心代码与测试结果 **2**

2.1 核心代码 2

2.1.1 构造函数 2

2.1.2 增删操作 3

2.1.3 打印操作 3

2.1.4 查找操作 4

2.2 测试结果 4

3 心得体会 **5**

4 加分项 **5**

5 附录：全部代码 **6**

实验目的与设计思路

1.1 实验目的

题目 1.1： 建立一个名单类 NameList，包括数据成员 std::vector<std::string> names，以及成员函数

1. 构造函数/析构函数，拷贝构造，赋值重载
2. void addName(const std::string&)，添加名字
3. void deleteName(const std::string&)，删除名字
4. std::vector<std::string> search(const std::string& substr)，查找包含字串的所有名字
5. void print(int order)，打印名字列表, order=0 表示按存储顺序打印，1 表示升序，2 表示降序
6. 编写主函数测试所有功能，手工加入 5 个以上的名字，如 Zhang San, Li Si，建议学习文件读写方法，实现数据的读入。

本实验主要测试了针对 **std::vector** 和 **std::string** 的基本操作，包括增删查改等功能。通过本实验，掌握 C++ 类的设计与实现方法，熟悉常用数据结构的操作技巧。

1.2 设计思路

本实验的设计思路如下：按照上述要求依次实现**类变量**，**构造/析构函数**，**成员函数**等模块。对于每个成员函数，先明确其功能与输入输出，然后编写代码实现。最后编写主函数进行测试，确保各功能正确实现。**std::vector** 作为 C++ 标准库中常用的动态数组容器，基本的接口有：**push_back** 尾插元素，**clear** 清空数组，同时可以用下标访问元素。其中由于涉及对字符串的查找与排序，采用了 C++ 标准库中的相关算法与函数，如 **std::find**、**std::sort** 等，以简化实现过程并提高效率。

核心代码与测试结果

2.1 核心代码

2.1.1 构造函数

由于构造函数的声明仅包含一个字符串，因此直接使用成员初始化列表进行初始化。同时，由于 **std::string** 默认即是创建一个新的地址复制，即深度复制，所以可以直接将别的 names 赋值给当前对象的 names。析构函数中调用 **clear()** 方法清空 names 向量，释放内存。

Listing 1: 构造函数与析构函数

```
NameList::NameList(std::string name) : names{std::move(name)} {}
NameList::~NameList() { names.clear(); }
NameList::NameList(const NameList& other) : names(other.names) {}
```

```
NameList& NameList::operator=(const NameList& other) {
    if (this != &other) { names = other.names; }
    return *this;
}
```

2.1.2 增删操作

添加名字时，直接调用 **push_back** 方法将名字添加到 names 向量末尾。删除名字时，先使用 **std::find** 查找名字在 names 中的位置，若找到则调用 **erase** 方法删除该名字，否则输出未找到提示。[1]

Listing 2: 增删操作

```
void NameList::addName(const std::string& newName) { this->names.push_back(newName); }
void NameList::deleteName(const std::string& deName) {
    auto iter  = std::find(names.begin(), names.end(), deName);
    while (iter != names.end()) {
        names.erase(iter);
        iter = std::find(names.begin(), names.end(), deName);
    }
}
```

2.1.3 打印操作

实验要求打印名字列表，需要支持多种顺序打印，包括原储存顺序，字典升序和字典降序。对于原储存顺序，直接遍历 names 向量输出即可。对于升序和降序，可以采用创建助手函数的方法，返回原储存向量的不同排序副本，然后遍历输出。对于排序，采用 C++ 标准库中的 **std::sort** 函数，传入不同的比较器实现升序和降序排序[2]。

Listing 3: 打印操作，部分代码

```
std::vector<std::string> increseOrder(const NameList& list) {
    std::vector<std::string> iterlist = list.getNames();
    std::sort(iterlist.begin(), iterlist.end());
    return iterlist;
}
bool cmp(std::string const a, std::string const b) {
    return a > b;
}
std::vector<std::string> decreaseOrder(const NameList& list) {
    std::vector<std::string> iterlist = list.getNames();
    std::sort(iterlist.begin(), iterlist.end(), cmp);
    return iterlist;
```

[1] https://blog.csdn.net/mashiromylove/article/details/137673545

[2] https://blog.csdn.net/2301_79396857/article/details/136129239

```
}
// order=0表示按存储顺序打印，1表示升序，2表示降序
void NameList::print(int order) {
    switch (order) {
        case 0: { ... }
        case 1: {
            for (auto name : increseOrder(*this)) {
                std::cout << name << ' ';
            }
            std::cout << std::endl;
            break;
        }
        case 2: { ... }
    }
}
```

2.1.4 查找操作

查找所有名字中包含指定子串的名字，采用遍历 names 向量的方法，使用 **std::string** 的成员函数 **find** 进行子串查找，若找到则将该名字添加到结果向量中，最后返回结果向量。[3]

Listing 4: 查找操作

```
std::vector<std::string> NameList::search(const std::string& substr) {
    std::vector<std::string> contains;
    for (auto name : names) {
        if (name.find(substr) != std::string::npos) { contains.push_back(name); }
    }
    return contains;
}
```

2.2 测试结果

对于测试部分，本实验采用自动化测试方案————**Google Test**[4]。Google Test 有简易的基础语法如 TEST()、EXPECT_EQ()、ASSERT_TRUE()，可以以此验证程序是否符合预期。
同时采用自动化测试，可以:

1. 自动检测程序逻辑错误，减少人工测试工作量；
2. 将测试与实现分离，提高程序的可维护性和可靠性。

在编写测试用例之前，先在 **names.txt** 文件中写入六个名字，作为测试数据。

[3] https://blog.csdn.net/llllllliuhz_/article/details/147228341
[4] https://google.github.io/googletest/primer.html

Listing 5: names.txt

```
Zhang san
Li si
Wang wu
Xiao hong
Da zhuang
Qi qi
```

本次测试用例分为了两个测试函数，分别测试初始化打印和增删查，思路为先从**.txt** 文件中读取数据[5]，然后进行相应操作并与预期结果对照验证正确性。限于篇幅不列出所有测试代码，得到测试结果为：

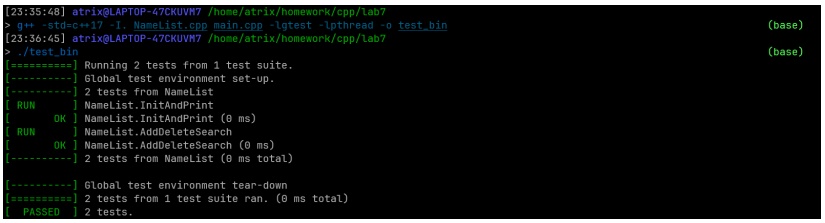

图 1: 测试结果

可以认为程序符合预期，是正确的。

心得体会

完成了本实验的要求，我有以下感悟：

- 多从利用网络搜索从标准库中寻找合适的算法与数据结构，能大大简化实现过程，提高代码效率与可读性；

加分项

本次实验除完成基本功能外，还在以下方面进行了扩展与优化，我认为可以视作加分项：

- 实现从文件中读取名字，同时对多处使用的标准库函数使用方式标注信息来源；
- 采用自动化测试（Google Test）：使用 Google Test 编写系统化测试用例，实现了对各函数逻辑的自动验证，体现了工程化测试思维；
- 接口与实现分离设计：将函数的声明与定义分别放置于.h 和.cpp 文件中，遵循模块化与可维护性原则。

[5] https://www.runoob.com/cplusplus/cpp-files-streams.html

附录：全部代码

Listing 6: NameList.h

```cpp
#ifndef NAMELIST_H
#define NAMELIST_H
#include <string>
#include <vector>
class NameList {
public:
    NameList(std::string name);
    ~NameList();
    NameList(const NameList&);
    NameList& operator=(const NameList&);
    void addName(const std::string&);
    void deleteName(const std::string&);
    std::vector<std::string> search(const std::string& substr);
    void print(int order);
    std::vector<std::string> getNames() const;
private:
    std::vector<std::string> names;
};
#endif
```

Listing 7: NameList.cpp

```cpp
#include "NameList.h"
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

NameList::NameList(std::string name) : names{std::move(name)} {
}

NameList::~NameList() {
    names.clear();
}

NameList::NameList(const NameList& other) : names(other.names) {
}

NameList& NameList::operator=(const NameList& other) {
    if (this != &other) {
```

```
        names = other.names;
    }
    return *this;
}

void NameList::addName(const std::string& newName) {
    this->names.push_back(newName);
}

void NameList::deleteName(const std::string& deName) { //采用find() + erase(),见https://blog.csdn.net/mashiromylove/article/details/137673545
    auto iter  = std::find(names.begin(), names.end(), deName);
    while (iter != names.end()) {
        names.erase(iter);
        iter = std::find(names.begin(), names.end(), deName);
    }
}

std::vector<std::string> NameList::getNames() const {
    return names;
}

std::vector<std::string> increseOrder(const NameList& list) { //辅助函数，升序排列，见https://blog.csdn.net/2301_79396857/article/details/136129239
    std::vector<std::string> iterlist = list.getNames();
    std::sort(iterlist.begin(), iterlist.end());
    return iterlist;
}

bool cmp(std::string const a, std::string const b) {
    return a > b;
}

std::vector<std::string> decreaseOrder(const NameList& list) {
    std::vector<std::string> iterlist = list.getNames();
    std::sort(iterlist.begin(), iterlist.end(), cmp);
    return iterlist;
}

// order=0表示按存储顺序打印，1表示升序，2表示降序
void NameList::print(int order) {
    switch (order) {
        case 0: {
            for (auto name : names) {
```

```
                std::cout << name << ' ';
            }
            std::cout << std::endl;
            break;
        }
        case 1: {
            for (auto name : increseOrder(*this)) {
                std::cout << name << ' ';
            }
            std::cout << std::endl;
            break;
        }
        case 2: {
            for (auto name : decreaseOrder(*this)) {
                std::cout << name << ' ';
            }
            std::cout << std::endl;
            break;
        }
    }

}

// 查找包含字串的所有名字，见https://blog.csdn.net/llllllliuhz_/article/details/147228341
std::vector<std::string> NameList::search(const std::string& substr) {
    std::vector<std::string> contains;
    for (auto name : names) {
        if (name.find(substr) != std::string::npos) {
            contains.push_back(name);
        }
    }
    return contains;
}
```

注：要查看检测信息，需要文件结构满足以下结构：

```
lab7/
|- NameList.h
|- NameList.cpp
|- main.cpp
|- names.txt
```

随后在终端输入

```
$ g++ -std=c++17 NameList.cpp main.cpp -I. -lgtest -lpthread -o test_bin
$ ./test_bin
```

Listing 8: names.txt

```
Zhang san
Li si
Wang wu
Xiao hong
Da zhuang
Qi qi
```

Listing 9: main.cpp

```cpp
#include "NameList.h"
#include <gtest/gtest.h>
#include <fstream>
#include <iostream>

TEST(NameList, InitAndPrint) {
    std::ifstream afile;
    afile.open("names.txt");
    if (!afile.is_open()) {
        std::cerr << "Failed to open names.txt" << std::endl;
    }
    std::string name;
    std::getline(afile, name);
    NameList namelist(name);
    while (std::getline(afile, name)) {
        namelist.addName(name);
    }
    afile.close();
    NameList namelist2 = namelist; //测试拷贝构造函数
    NameList namelist3(namelist);
    testing::internal::CaptureStdout();
    namelist.print(0);
    std::string output = testing::internal::GetCapturedStdout();
    EXPECT_EQ("Zhang san Li si Wang wu Xiao hong Da zhuang Qi qi \n", output);
    testing::internal::CaptureStdout();
    namelist2.print(1);
    output = testing::internal::GetCapturedStdout();
    EXPECT_EQ("Da zhuang Li si Qi qi Wang wu Xiao hong Zhang san \n", output);
    testing::internal::CaptureStdout();
    namelist3.print(2);
    output = testing::internal::GetCapturedStdout();
    EXPECT_EQ("Zhang san Xiao hong Wang wu Qi qi Li si Da zhuang \n", output);
```

```cpp
}


TEST(NameList, AddDeleteSearch) {
    std::ifstream afile;
    afile.open("names.txt");
    if (!afile.is_open()) {
        std::cerr << "Failed to open names.txt" << std::endl;
    }
    std::string name;
    std::getline(afile, name);
    NameList namelist(name);
    while (std::getline(afile, name)) {
        namelist.addName(name);
    }
    afile.close();
    namelist.deleteName("Li si");
    std::vector<std::string> names = namelist.getNames();
    std::vector<std::string> expected_names = {"Zhang san", "Wang wu", "Xiao hong", "Da zhuang",
    "Qi qi"};
    EXPECT_EQ(names, expected_names);
    namelist.addName("Xiao ming");
    std::vector<std::string> updated_names = namelist.getNames();
    std::vector<std::string> expected_updated_names = {"Zhang san", "Wang wu", "Xiao hong", "Da
    zhuang", "Qi qi", "Xiao ming"};
    EXPECT_EQ(updated_names, expected_updated_names);
    std::vector<std::string> search_result = namelist.search("an");
    std::vector<std::string> expected_search_result = {"Zhang san", "Wang wu", "Da zhuang"};
    EXPECT_EQ(search_result, expected_search_result);
}

int main(int argc, char **argv) {
    ::testing::InitGoogleTest(&argc, argv);
    return RUN_ALL_TESTS();
}


// int main() {
//     std::ifstream afile;
//     afile.open("names.txt");
//     if (!afile.is_open()) {
//         std::cerr << "Failed to open names.txt" << std::endl;
//         return 1;
//     }
```

```
//     std::string name;
//     std::getline(afile, name);
//     NameList namelist(name);
//     while (std::getline(afile, name)) {
//         namelist.addName(name);
//     }
//     afile.close();
//     namelist.print(0);
//     namelist.print(1);
//     namelist.print(2);
//     return 0;
// }
```